

APACHE AIRFLOW TRAINING

Hands-On Lab Guide

Getting Started with Astro CLI and Apache Airflow

Install · Initialize · Run · Deploy

Environment: AWS EC2 Ubuntu + Docker Compose Airflow

Level: Beginner to Intermediate | Duration: ~90 minutes

Tool: Astronomer Astro CLI | Airflow Version: 2.x

Table of Contents

1. Lab Overview & Objectives
2. Install Astro CLI on Ubuntu
 - 2.1 Prerequisites
 - 2.2 Install via curl (Official Method)
 - 2.3 Verify Installation
3. Initialize an Astro Project
 - 3.1 Create Project Directory
 - 3.2 Run astro dev init
 - 3.3 Explore Project Structure
4. Run Airflow Locally with Astro
 - 4.1 Start the Astro Dev Environment
 - 4.2 Verify Running Containers
 - 4.3 Access the Airflow UI
5. Deploy Your First DAG
 - 5.1 Understand the Example DAG
 - 5.2 Write a Custom DAG
 - 5.3 Validate and Deploy
 - 5.4 Monitor in the UI
6. Working with the Astro Project
 - 6.1 Project File Reference
 - 6.2 Managing Dependencies
 - 6.3 Stopping and Restarting
7. Troubleshooting
8. Summary & Next Steps

Section 1

Lab Overview & Objectives

This lab introduces the **Astronomer Astro CLI** — the recommended developer tool for building, testing, and deploying Apache Airflow projects. Even though your EC2 instance already runs Airflow via Docker Compose, the Astro CLI gives you a production-grade, project-based workflow with one-command environment management.

■ Who is this lab for?

Participants who have an Airflow environment running on AWS EC2 Ubuntu (via Docker Compose) and want to adopt a professional, CLI-driven Airflow development workflow.

✓ Objective 1	Install the Astro CLI on an Ubuntu EC2 instance
✓ Objective 2	Initialise a new Astro project with the correct folder structure
✓ Objective 3	Start a fully isolated local Airflow environment using Astro
✓ Objective 4	Write and deploy a working DAG to the local Airflow UI
✓ Objective 5	Understand the Astro project layout and key configuration files

Environment Assumptions

- AWS EC2 instance running **Ubuntu 22.04 LTS** (t2.medium or larger)
- Docker and Docker Compose already installed and running
- Existing Airflow Docker Compose stack is UP (webserver, scheduler, worker, postgres, redis)
- Ports 8080 (Airflow UI) and 8081 (Astro Airflow UI) are open in the Security Group
- You are connected via SSH with a non-root user that has **sudo** privileges

■ Port Note

The existing Docker Compose Airflow already uses port 8080. When you start the Astro dev environment it will use port 8080 by default. You can either stop the existing stack first OR configure Astro to use a different port (covered in Section 4).

Section 2

Install Astro CLI on Ubuntu

2.1 Prerequisites

Before installing the Astro CLI, verify that the following tools are available on your EC2 instance.

Step 1 — Check Docker version

Terminal

```
$ docker --version
$ docker compose version
```

■ Expected Output

```
Docker version 24.0.5, build ced0996
Docker Compose version v2.20.3
```

✗ Docker not found?

If docker is not found, contact your lab administrator. Docker must be running before you proceed.

Step 2 — Check available disk space

Terminal

```
$ df -h /
```

Ensure at least **4 GB free** in / before continuing. The Astro CLI pulls Docker images (~2.5 GB).

2.2 Install via curl (Official Method)

Astronomer provides a shell script installer that handles the binary download, checksum verification, and PATH setup automatically.

Step 3 — Download and run the Astro CLI installer

Terminal

```
$ curl -sSL install.astronomer.io | sudo bash -s
```

■ Expected Output

```
Retrieving Astro CLI...
Installing Astro CLI v1.x.x
Astro CLI installed to /usr/local/bin/astro
Run 'astro version' to confirm the installation.
```

■ What does this script do?

The installer script downloads the latest stable Astro CLI binary for Linux/amd64, verifies the SHA256 checksum, copies the binary to `/usr/local/bin/astro`, and sets the executable bit. No package manager is modified.

Step 4 — Alternative: Install a specific version

If you need a specific version for reproducibility, pass the version tag to the script.

Terminal

```
# Install specific version
$ curl -sSL install.astronomer.io | sudo bash -s -- --version 1.25.0
```

2.3 Verify Installation

Step 5 — Confirm the CLI is accessible

Terminal

```
$ astro version
```

Expected Output

```
Astro CLI Version: 1.25.0
Git commit hash: abcd1234
```

Step 6 — Explore available commands

Terminal

```
$ astro --help
```

Expected Output

```
The official Astronomer CLI for Apache Airflow

Usage:
astro [command]

Available Commands:
completion Generate the autocompletion script
config Manage Astro project configuration
deploy Deploy code to Astro
dev Run your Astro project in a local Airflow environment
version Print the current Astro CLI version
workspace Manage Astro workspaces
...
```

◆ Enable shell auto-completion

For a better developer experience, enable tab-completion by adding the following to your `~/.bashrc`: `eval "$(astro completion bash)"` Then run: `source ~/.bashrc`

Section 3

Initialize an Astro Project

An **Astro project** is a standardized directory structure that packages all the components needed to run Apache Airflow — DAGs, plugins, requirements, environment variables, and a Dockerfile. Running `astro dev init` scaffolds this structure for you instantly.

3.1 Create a Project Directory

Step 1 — Create and enter a new directory for your Astro project

Terminal

```
$ mkdir ~/my-airflow-project  
$ cd ~/my-airflow-project
```

■ Naming convention

Use lowercase letters, hyphens, or underscores. Avoid spaces. This directory will become the root of your Astro project.

3.2 Run astro dev init

Step 2 — Initialise the Astro project

Terminal

```
$ astro dev init
```

■ Expected Output

```
Initializing Astro project...  
  
_ _ _  
/ _ \ _ _ _ | | _ _ _  
| | | | / _ \ | | _ \ _ \  
| _ _/ | ( _ | | | | ( _ |  
| _ \ _ , _ \ _ | _ \ _  
  
astro dev init completed.  
  
Your Astro project has been successfully initialized!  
If this is your first time using Astro, start with:  
astro dev start
```

3.3 Explore the Project Structure

Step 3 — List all generated files and directories

Terminal

```
$ ls -la
$ find . -type f | sort
```

■ Project Structure

```
my-airflow-project/
  ■■■ .astro/
  ■ ■■■ config.yaml # Astro CLI project configuration
  ■■■ dags/
  ■ ■■■ exampledag.py # Starter example DAG
  ■■■ include/ # Custom modules, SQL files, etc.
  ■■■ plugins/ # Custom Airflow plugins
  ■■■ tests/
  ■ ■■■ dags/
  ■ ■■■ test_dag_integrity.py
  ■■■ .dockerignore
  ■■■ .env # Local environment variables
  ■■■ .gitignore
  ■■■ airflow_settings.yaml # Pre-defined connections, variables, pools
  ■■■ Dockerfile # Custom Airflow image definition
  ■■■ requirements.txt # Python dependencies
```

Understanding each file is critical for productive Airflow development. The table below describes the purpose of each key file.

Dockerfile	Extends the official Astronomer Runtime image. Add OS-level dependencies here (e.g., apt-get install).
requirements.txt	Python packages installed via pip into the Airflow environment. Add provider packages here.
.env	Environment variables injected at runtime. Never commit secrets here — use a secrets backend in production.
airflow_settings.yaml	Defines Connections, Variables, and Pools to be auto-seeded when the environment starts. Great for local dev.
dags/	Drop your DAG .py files here. Airflow scans this folder automatically.
include/	Store helper modules, SQL scripts, config files, and other assets referenced by your DAGs.
plugins/	Custom Airflow plugins: custom operators, hooks, sensors, executors.
.astro/config.yaml	Project-level Astro CLI configuration: project name, default registry, etc.
tests/dags/	Unit tests. The default test_dag_integrity.py checks every DAG for import errors and cycle-free structure.

Step 4 — Inspect the example DAG

Terminal

```
$ cat dags/exampledag.py
```

❖ Open in editor

If you prefer a graphical editor, use VS Code with the Remote-SSH extension to browse and edit files on your EC2 instance directly from your laptop.

Section 4

Run Airflow Locally with Astro

The `astro dev start` command builds a custom Docker image from your project's Dockerfile, then orchestrates a complete Airflow environment (webserver, scheduler, triggerer, and Postgres metadata DB) using Docker Compose — all managed transparently by the Astro CLI.

4.1 Handle the Existing Docker Compose Airflow (Port Conflict)

■ Port 8080 conflict

Your EC2 instance already runs Airflow on port 8080 via Docker Compose. If you try to start Astro without addressing this, you will get a port bind error. Choose ONE of the two options below.

Step 1 — Option A — Stop the existing Airflow stack first (Recommended for lab)

Terminal

```
# Navigate to where your existing docker-compose.yml is
$ cd ~/airflow # adjust path as needed
$ docker compose down

# Confirm no containers remain on port 8080
$ docker ps

# Return to your Astro project
$ cd ~/my-airflow-project
```

Step 2 — Option B — Use a different port for Astro (Keep existing Airflow running)

Terminal

```
# Start Astro on port 8081 instead
$ astro dev start --wait 3m -e AIRFLOW__WEBSERVER__BASE_URL=http://localhost:8081

# OR set the port via .env file
$ echo 'AIRFLOW__WEBSERVER__EXPOSE_PORT=8081' >> .env
$ astro dev start
```

4.2 Start the Astro Dev Environment

Step 3 — Start the local Airflow environment

Terminal

```
$ cd ~/my-airflow-project
$ astro dev start
```

■ Expected Output

```
Sending build context to Docker daemon ...
Step 1/1 : FROM quay.io/astronomer/astro-runtime:11.0.0
11.0.0: Pulling from astronomer/astro-runtime
...
Successfully built 4f3d7a2c1e89
Successfully tagged my-airflow-project/airflow:latest
INFO[0001] Starting Airflow project: my-airflow-project
INFO[0001] Starting the webserver on port 8080...
INFO[0001] Starting the scheduler...
INFO[0001] Starting the triggerer...

Airflow is starting up!

Project: my-airflow-project
Status: Running
Webserver: http://localhost:8080
Postgres: localhost:5432
Username: admin
Password: admin

Congrats! Airflow is up and running.
Use astro dev stop to stop the environment.
```

■ First run takes longer

On the first run, Docker pulls the Astro Runtime base image (~800 MB). Subsequent starts are much faster because the image is cached locally. Expect 3–8 minutes on first run depending on network speed.

4.2 Verify Running Containers

Step 4 — Check that all Astro containers are healthy

Terminal

```
$ docker ps
```

■ Expected Output

```
CONTAINER ID IMAGE COMMAND STATUS
a1b2c3d4e5f6 my-airflow-project/airflow 'webserver' Up 2 min (healthy)
b2c3d4e5f6a7 my-airflow-project/airflow 'scheduler' Up 2 min (healthy)
c3d4e5f6a7b8 my-airflow-project/airflow 'triggerer' Up 2 min (healthy)
d4e5f6a7b8c9 postgres:16.0-bullseye 'postgres' Up 2 min (healthy)
```

Step 5 — Check Astro project status

Terminal

```
$ astro dev ps
```

■ Expected Output

```
Name State
my-airflow-project_webserver Running
my-airflow-project_scheduler Running
my-airflow-project_triggerer Running
my-airflow-project_postgres Running
```

4.3 Access the Airflow UI

Step 6 — Open the Airflow Web UI in your browser

From your local machine, open a browser and navigate to:

Browser URL

```
http://:8080

# Example:
http://54.123.45.67:8080
```

Username	admin
Password	admin

■ EC2 Security Group

Ensure your EC2 Security Group inbound rules allow TCP port 8080 from your IP address (or 0.0.0.0/0 for lab purposes). Without this rule, the browser will time out.

Step 7 — Verify Astro logs (optional)

Terminal

```
# Stream all container logs
$ astro dev logs

# Webserver logs only
$ astro dev logs --webserver

# Scheduler logs only
$ astro dev logs --scheduler
```


◆ TaskFlow API

This DAG uses the modern **TaskFlow API** (`@dag` and `@task` decorators), introduced in Airflow 2.0. XCom data passing between tasks happens automatically — you don't need to call `xcom_push`/`xcom_pull` manually.

5.3 Validate the DAG Before Deployment

Step 4 — Check the DAG file for Python syntax errors

Terminal

```
$ python3 -c "import ast; ast.parse(open('dags/my_first_dag.py').read()); print('Syntax OK')"
```

■ Expected Output

```
Syntax OK
```

Step 5 — Run the Astro built-in DAG tests

Terminal

```
$ astro dev pytest

# This runs the tests/dags/test_dag_integrity.py which checks:
# - All DAGs import without errors
# - No cycles in task dependencies
# - DAG tags are present
```

■ Expected Output

```
===== test session starts =====
platform linux -- Python 3.11.5
collected 2 items

tests/dags/test_dag_integrity.py ..

===== 2 passed in 3.12s =====
```

✗ Import errors in DAG test?

If the test fails with an `ImportError`, check that all packages used in your DAG are either built into Airflow or listed in `requirements.txt`. After editing `requirements.txt`, run `'astro dev restart'` to rebuild.

Step 6 — Parse the DAG via the Airflow CLI (inside the container)

Terminal

```
$ astro dev run dags list

# Should show your new DAG in the list
```

■ Expected Output

```
dag_id filepath owner paused
=====
example_astronauts dags/exampledag.py airflow False
my_first_astro_dag dags/my_first_dag.py astro-trainee False
```

5.4 Monitor the DAG in the Airflow UI

Step 7 — Open the Airflow UI and find your DAG

- Navigate to http://<EC2_IP>:8080 in your browser
- Log in with **admin / admin**
- On the DAGs list page, search for `my_first_astro_dag`
- Click the toggle to **Unpause** the DAG (toggle left of the DAG name)

Step 8 — Trigger a manual DAG run

- Click the **■ (Trigger DAG)** button on the right side of the DAG row
- Click **Trigger** in the confirmation dialog (leave config empty)
- Click the DAG name to open the **Grid View**
- Watch the run appear — squares turn **green** when tasks succeed

Step 9 — Inspect task logs

- In Grid View, click any **colored square** (a task instance)
- Click **Logs** in the popup panel
- You should see the `logging.info()` messages from your Python functions
- Verify the temperature conversion: $28.5^{\circ}\text{C} = 83.3^{\circ}\text{F}$

■ Task Log Output

```
[2024-01-15 10:22:05,123] {logging_mixin.py:188} INFO - Extracted data: {'temperature': 28.5, 'city': 'Hyderabad', 'unit': 'C'}
[2024-01-15 10:22:07,456] {logging_mixin.py:188} INFO - Transformed data: {'temperature': 28.5, 'city': 'Hyderabad', 'unit': 'C', 'temperature_f': 83.3}
[2024-01-15 10:22:09,789] {logging_mixin.py:188} INFO - --- LOAD COMPLETE ---
[2024-01-15 10:22:09,790] {logging_mixin.py:188} INFO - temperature: 28.5
[2024-01-15 10:22:09,791] {logging_mixin.py:188} INFO - city: Hyderabad
[2024-01-15 10:22:09,792] {logging_mixin.py:188} INFO - temperature_f: 83.3
```

◆ Auto-refresh

The Airflow UI does not auto-refresh by default. Click the **■ Auto-refresh** toggle in the top-right of Grid View to enable live updates.

Section 6

Working with the Astro Project

6.1 Project File Reference

This section provides a practical reference for the most common day-to-day tasks you will perform in your Astro project.

6.2 Managing Python Dependencies

Step 1 — Add a Python package to requirements.txt

requirements.txt

```
# Edit requirements.txt
$ nano requirements.txt

# Add provider packages, e.g.:
apache-airflow-providers-postgres==5.11.0
apache-airflow-providers-http==4.10.0
pandas==2.1.4
requests==2.31.0
```

Step 2 — Rebuild the image to install new packages

Terminal

```
# Stop and rebuild (required after requirements.txt changes)
$ astro dev restart

# OR stop then start
$ astro dev stop
$ astro dev start
```

■ Always restart after requirements.txt changes

Changes to requirements.txt are not picked up at runtime — they require a Docker image rebuild. Use 'astro dev restart' to trigger a rebuild. DAG file changes, however, are picked up live without restart.

6.2 Seeding Connections and Variables

Step 3 — Use airflow_settings.yaml to pre-configure connections

Terminal

```
# Edit airflow_settings.yaml
```

```
$ nano airflow_settings.yaml
```

airflow_settings.yaml

```
airflow:
connections:
- conn_id: my_postgres_conn
  conn_type: postgres
  conn_host: localhost
  conn_schema: mydb
  conn_login: airflow
  conn_password: airflow
  conn_port: 5432

variables:
- variable_name: env
  variable_value: development

pools:
- pool_name: default_pool
  pool_slot: 128
  pool_description: Default pool
```

■ When does airflow_settings.yaml take effect?

Connections and variables defined in airflow_settings.yaml are imported automatically when 'astro dev start' runs. To re-import after changes, run 'astro dev restart'.

6.3 Stopping and Restarting the Environment

astro dev start	Start the Airflow environment (builds image if needed)
astro dev stop	Stop all containers (preserves data in volumes)
astro dev restart	Stop + rebuild image + start (use after requirements.txt changes)
astro dev kill	Force-stop and remove all containers AND volumes (full reset)
astro dev ps	List running containers and their status
astro dev logs	Stream logs from all containers
astro dev bash	Open a bash shell inside the scheduler container
astro dev run	Run an Airflow CLI command inside the scheduler container

`astro dev pytest`

Run DAG integrity tests

Step 4 — Execute an Airflow CLI command via Astro

Terminal

```
# List all DAGs
$ astro dev run dags list

# Manually trigger a DAG run
$ astro dev run dags trigger my_first_astro_dag

# List all connections
$ astro dev run connections list

# Test a specific task
$ astro dev run tasks test my_first_astro_dag extract 2024-01-01
```

Step 5 — Open a shell inside the scheduler container

Terminal

```
$ astro dev bash

# You are now inside the scheduler container
# Useful for debugging imports or checking the environment
(container) $ python -c 'import pandas; print(pandas.__version__)'
(container) $ airflow info
(container) $ exit
```

Section 7

Troubleshooting

X Error: port 8080 is already allocated

The existing Docker Compose Airflow stack is using port 8080.

Fix

```
$ cd ~/airflow # navigate to your existing compose directory
$ docker compose down
$ cd ~/my-airflow-project
$ astro dev start
```

X astro: command not found after installation

/usr/local/bin is not in your PATH, or you need to reload your shell.

Fix

```
$ export PATH=$PATH:/usr/local/bin
$ echo 'export PATH=$PATH:/usr/local/bin' >> ~/.bashrc
$ source ~/.bashrc
$ astro version
```

X DAG not appearing in the UI

Airflow scans the dags/ folder every 30 seconds by default. If the DAG has an import error, it will not appear.

Fix

```
# Check for import errors
$ astro dev run dags list-import-errors

# Parse the specific file
$ astro dev bash
(container) $ python dags/my_first_dag.py
```

X ModuleNotFoundError in task logs

A Python package used in the DAG is not installed in the image.

Fix

```
# Add the package to requirements.txt
```

```
$ echo 'requests==2.31.0' >> requirements.txt

# Rebuild the image

$ astro dev restart
```

X Webserver shows 'DAG Broken' badge

The DAG file has an import error or syntax issue.

Fix

```
$ astro dev run dags list-import-errors

# Fix the error in the DAG file, it reloads automatically
```

X astro dev start hangs or times out

Docker image pull is slow, or the webserver health check is timing out.

Fix

```
# Increase the wait timeout

$ astro dev start --wait 10m

# Check container status in another terminal

$ docker ps

$ docker logs
```

Section 8

Summary & Next Steps

Congratulations! You have successfully completed the core Astro CLI workflow:

✓ Installed	Astro CLI on Ubuntu EC2 using the official installer script
✓ Initialized	A structured Astro project with <code>astro dev init</code>
✓ Started	A fully isolated Airflow environment with <code>astro dev start</code>
✓ Deployed	A custom ETL DAG using the TaskFlow API
✓ Monitored	The DAG run in the Airflow UI and verified task logs

Recommended Next Steps

- **Add more operators** — Try `BashOperator`, `PythonOperator`, `EmailOperator` in your DAG
- **Install a provider** — Add `apache-airflow-providers-postgres` to `requirements.txt` and create a database connection
- **Use Sensors** — Add a `FileSensor` or `HttpSensor` to make your pipeline data-driven
- **Schedule variations** — Experiment with cron expressions: `'0 6 * * *'` (daily at 6 AM), `'0 */4 * * *'` (every 4 hours)
- **Dataset scheduling** — Explore Airflow 2.4+ Dataset-based scheduling for event-driven pipelines
- **Deploy to Astronomer Cloud** — When ready for production, run `astro login` and `astro deploy`
- **Explore the Astronomer Registry** — `registry.astronomer.io` has 1000+ provider integrations and DAG templates

◆ Astro CLI Cheat Sheet (Quick Reference)

`astro dev init` → Scaffold a new project `astro dev start` → Start local Airflow `astro dev stop` → Stop local Airflow `astro dev restart` → Rebuild image + restart `astro dev kill` → Full reset (removes volumes) `astro dev logs` → Stream container logs `astro dev bash` → Shell into scheduler `astro dev run` → Run Airflow CLI command `astro dev pytest` → Run DAG integrity tests `astro dev parse` → Parse DAGs for errors

End of Lab Guide — Apache Airflow with Astro CLI
Training Material | AWS EC2 + Docker Compose Environment